

# NLP on Real Time Systems

Jackson Grizzle  
Dept. of Computer Science  
University of Houston  
Houston, TX 77004, USA  
jacksondgrizzle@proton.me

**Abstract**—While Natural Language Processing (NLP) allows for sophisticated human-machine interaction, the inherent non-determinism of high-level languages and the complex nature of neural networks makes Recurrent Neural Network (RNN) models unfeasible for time sensitive systems. This paper evaluates three distinct RNN models, focusing on their implementation for time-sensitive environments. By transitioning from a Python-based environment to a real-time C++ environment, utilizing ONNX Runtime, and applying kernel-level optimizations, such as thread affinity and memory locking, a significant reduction in jitter can be reached. Experimental results after training on the SNIPS dataset indicate that the best proposed RNN configuration achieves a 96% reduction in Worst-Case Execution Time (WCET) and a 97% reduction in jitter compared to standard implementations, successfully making NLP deterministic enough for time-critical systems.

## I. INTRODUCTION

In recent years, voice-user interfaces (VUI) are no longer just for general-purpose virtual assistants, they are being integrated into autonomous vehicles and industrial safety systems. These systems originally used cloud NLP, however, this is infeasible for many newer systems. On-device processing guarantees privacy, lower bandwidth usage, and lower latency, while helping to increase portability. A small amount of jitter is not detrimental on soft-safety critical systems, where missing a deadline occasionally is acceptable, however, on a hard-safety critical system, misclassifying an intent or missing a deadline could have serious consequences, leading to real world damage or system failure. The current standard environment for creating and running neural networks (such as RNN models) is python. This is sufficient for research, however, python is not feasible in hard-safety critical systems. The global interpreter lock (GIL) only allows one thread to run python bytecode at a time. This includes the interpreter, garbage collection, and everything else python requires to run a program. Although this is sufficient in most high-level application logic, these slow-downs are unacceptable in a WCET study. In addition to this, default modern operating system configurations are optimized for throughput; getting more tasks done faster. In order to run a neural network on a time sensitive system, the OS must be optimized for latency; getting the specific task done quickly. Interrupts, page faults, and context switching can cause jitter, which is not ideal in these circumstances. This paper proposes a high-performance deterministic inference pipeline using C++, ONNX Runtime, and kernel-level optimizations to mitigate these challenges.

## II. SYSTEM ARCHITECTURE

### A. Temporal Metrics

For the purpose of this analysis, the following temporal metrics are defined: Latency ( $L_{avg}$ ), is defined as the mean execution time across  $N$  iterations, ( $Mean(L)$ ). Jitter ( $J$ ) is defined as the variance between the WCET and Best-Case Execution Time (BCET), or  $J = L_{max} - L_{min}$

### B. Model Design and Selection

Three distinct Recurrent Neural Network (RNN) models were evaluated: a single-layer gated recurrent unit (GRU), a long short-term memory (LSTM) model, and a dual-layer GRU featuring an "early exit" mechanism. The single-layer GRU model was selected as the optimal candidate for the proposed real-time system. Compared to LSTMs, GRU's architecture utilizes fewer gates and lacks a separate cell state, which reduces the total number of matrix operations per timestep. This directly correlates to a reduction in both average latency and jitter.

Although the "early exit" model attempted to optimize the trade-off between a dual-layer GRU's accuracy and a single-layer's speed, it introduced significant non-determinism. Because the exit condition depends on a dynamic confidence threshold, the execution path is data-dependent, leading to bimodal latency distributions. Given that the single-layer GRU achieved a satisfactory  $F_1$ -score of 0.97, the additional complexity and jitter of the early exit model were deemed incompatible with hard real-time requirements. To ensure determinism through uniform inputs, all models utilize a fixed input sequence length of 15 tokens. Shorter inputs are zero-padded, and longer inputs are truncated, ensuring a constant number of operations for every inference cycle regardless of linguistic complexity.

### C. Training and Portability

The models were developed and trained in PyTorch, using the SNIPS NLU dataset. Upon reaching a performance threshold ( $F_1$ -score  $\geq 0.95$ ), the models were exported into Open Neural Network Exchange (ONNX) models. Transitioning from a dynamic Python-based execution environment to a pre-compiled static ONNX representation in C++ allows the system to bypass the overhead of the Python interpreter and the GIL and GC. Running the models in C++ also allows for kernel-level optimizations, which will be detailed in the next section.

### III. REAL TIME IMPLEMENTATION

#### A. Task Scheduling

This project will be done on a Linux system, since Linux allows for easy kernel modification. The Linux kernel has a feature called PREEMPT\_RT, which implements both hard and soft real-time computing capabilities. Utilizing this feature, the conditions of an embedded real time system can be simulated. By default, the Linux kernel uses the Completely Fair Scheduler (CFS), which is designed to maximize throughput rather than individual task latency. To ensure the intent classifier is never preempted by background system processes, the scheduling policy was modified to SCHED\_FIFO (First-In, First-Out) using the `sched_setscheduler` API. A priority of 80 was assigned to the task. Priorities on Linux range from 1-99, so a priority of 80 ensures that most background processes will not preempt the intent classifier.

#### B. Memory Management

Since Linux memory management utilizes "demand paging," where physical memory frames are not allocated until a process attempts to access a virtual memory address, cache misses can lead to small spikes in latency. To avoid this, `mlockall(MCL_CURRENT | MCL_FUTURE)` is used. This pins the entire process and all of the ONNX data needed for it into the memory, before the intent classifier even runs. Furthermore, the stack is initialized with a large empty buffer using `memset`, so more memory does not have to be allocated at runtime, reducing jitter and latency even more.

#### C. Hardware Isolation

The last hardware optimization used to reduce jitter was CPU core isolation. This was not only to help decrease jitter, but was also done in order to simulate a smaller real time system, which might not have multiple cores for the intent classifier to run. By setting `isolcpus=3` in the system's GRUB boot configuration, a core can be isolated, where `pthread_setaffinity_np` can be used in the program and `taskset -c 3` can be used in the run configuration to bind the intent classifier to the isolated core.

### IV. EXPERIMENT AND RESULTS

#### A. Experiment Setup

The experiment is being conducted on an HP ENVY x360 Laptop with an Intel i7 processor and 32gb of ram running Arch Linux, however, the code can be implemented on most systems running Linux. The software used is Python 3.14, libonnxruntime version 3.4.30, and PyTorch 2.11.0. Each model was run 1400 times, using data from the SNIPS testing dataset.

#### B. Results

As demonstrated in Table I, the C++ implementation consistently outperformed the Python baseline across all three architectures. While the average latency of the GRU model was reduced by approximately 66%, the most significant

TABLE I  
COMPARATIVE PERFORMANCE AND DETERMINISM METRICS

| Model & Environment    | Mean Latency (ms) | WCET ( $L_{max}$ ) | Jitter (J) | Std. Dev. ( $\sigma$ ) |
|------------------------|-------------------|--------------------|------------|------------------------|
| <b>Python Baseline</b> |                   |                    |            |                        |
| LSTM                   | 0.69              | 4.53               | 4.17       | 0.27                   |
| GRU                    | 0.64              | 7.27               | 6.97       | 0.34                   |
| Early Exit             | 1.45              | 12.29              | 11.48      | 0.53                   |
| <b>C++ Optimized</b>   |                   |                    |            |                        |
| LSTM                   | 0.23              | 0.49               | 0.34       | 0.04                   |
| GRU                    | 0.22              | 0.38               | 0.24       | 0.03                   |
| Early Exit             | 0.4               | 0.7                | 0.43       | 0.06                   |

finding is the reduction in WCET. In the Python environment, the maximum recorded latency reached 12.29 ms with the early exit model, likely due to background garbage collection and interpreter overhead. In contrast, the C++ implementation maintained a WCET of 0.7 ms with the early exit model, representing a 94% improvement in temporal reliability.

#### C. Analysis

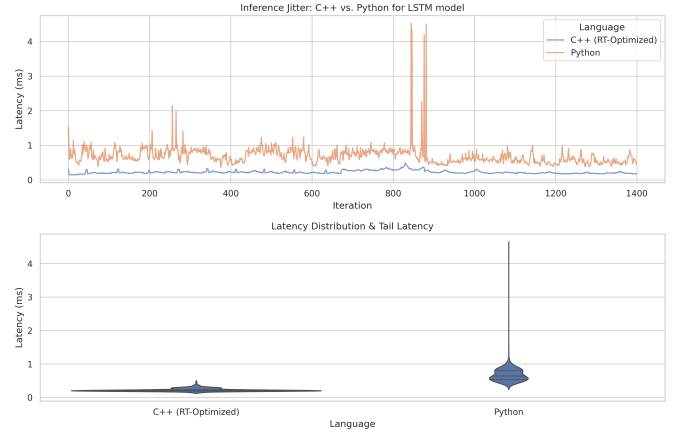


Fig. 1.

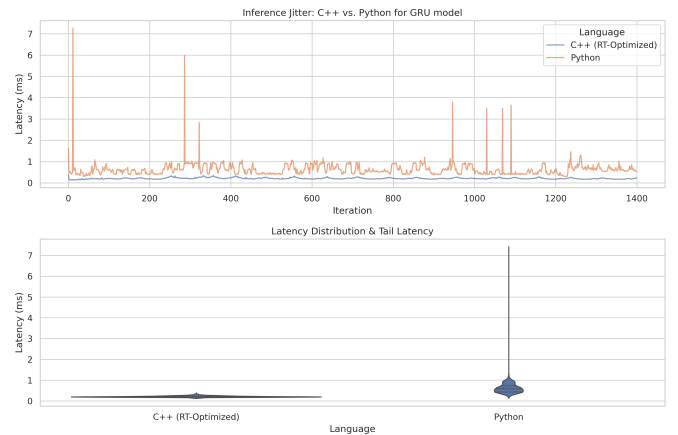


Fig. 2.

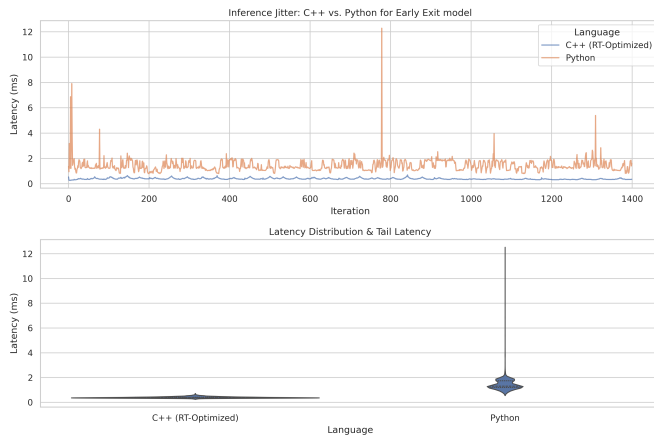


Fig. 3.

The visual analysis provided in Figures 1, 2, and 3 further reinforce these findings. The Python implementation shows significant and unpredictable latency spikes across all three models, characteristic of non-deterministic scheduling. In contrast, the C++ implementation effectively mitigates these spikes, demonstrating determinism. Specifically, for the Optimized GRU model, the jitter was reduced by 97%, decreasing from 6.97 ms in the python environment to 0.24 ms. This confirms that the proposed kernel-level optimizations successfully established a tight execution bound suitable for hard real-time constraints

## V. CONCLUSION

This paper has demonstrated that the perceived limitations of integrating RNN and neural networks into hard real-time systems are largely a consequence of the execution environment rather than the complexity of the architectures themselves. By transitioning from a high-level Python-based environment to an optimized C++ pipeline utilizing ONNX Runtime, we successfully mitigated the non-deterministic overhead introduced by interpreter management and standard OS scheduling.

Experimental results indicate that while the LSTM and Early Exit GRU models offer specific advantages in memory or average-case speed, the Optimized GRU provides the most reliable execution profile for safety-critical tasks. The implementation of core isolation, memory locking, and real-time scheduling resulted in a 97% reduction in jitter, lowering the Worst-Case Execution Time (WCET) bound to a deterministic 0.38 ms. This level of predictability ensures that intent classification, such as emergency stop commands, can be executed within strict deadlines..

Future research will explore the performance of this method on different embedded systems, and integration of the intent classifier with other systems, such as voice processing, control systems, and more. The research will focus on keeping determinism and optimizing latency further.

## REFERENCES

- [1] Chowdhary, KR1442. "Natural language processing." *Fundamentals of artificial intelligence* (2020): 603-649.
- [2] GeeksforGeeks, "Introduction to Natural Language Processing (NLP)," GeeksforGeeks, Jul. 14, 2021. <https://www.geeksforgeeks.org/nlp/introduction-to-natural-language-processing-nlp/>
- [3] GeeksforGeeks, "Gated Recurrent Unit Networks," GeeksforGeeks, Jul. 09, 2019. <https://www.geeksforgeeks.org/machine-learning/gated-recurrent-unit-networks/>
- [4] "Understanding Python's GIL and How It Affects Performance," w3tutorials, 2025. <https://www.w3tutorials.net/python-tutorial/understanding-pythons-gil-and-how-it-affects-performance/> (accessed May 07, 2026).
- [5] "Training Dataset Format — Snips NLU 0.20.2 documentation," Readthedocs.io, 2021. <https://snips-nlu.readthedocs.io/en/latest/dataset.html> (accessed May 07, 2026). capitalized," J. Name Stand. Abbrev., in press.
- [6] "Introduction to ONNX - ONNX 1.17.0 documentation," onnx.ai. <https://onnx.ai/onnx/intro/>
- [7] PyTorch Contributors, "PyTorch documentation," Pytorch.org, 2023. <https://docs.pytorch.org/docs/2.11/index.html>
- [8] "Real-time preemption — The Linux Kernel documentation," Kernel.org, 2026. <https://docs.kernel.org/core-api/real-time/> (accessed May 08, 2026).

All of the code for this project is located at <https://github.com/TacoJGriz/NLP-realtime>